NedoTurkishTokenizer: A Morphology-Aware Tokenizer and Human-Adjudicated Benchmark for Turkish

Nedim Mutlu Sezer
Ethosoft Research Group

LRE full-length preparation draft, 7 July 2026

Abstract

Turkish morphology is highly productive: a single orthographic word can encode a stem and a sequence of inflectional or derivational suffixes. Standard subword tokenizers are useful engineering tools, but their segmentation boundaries are not designed to coincide with linguistically meaningful Turkish morpheme boundaries. This paper presents NedoTurkishTokenizer, a self-contained morphology-aware tokenizer for Turkish, together with a 1,000-item human-adjudicated benchmark for evaluating Turkish tokenization boundaries and labels. The final gold set contains 754 annotator-agreed items, 50 previously adjudicated items, and 196 newly adjudicated items, with zero build issues. On this benchmark, NedoTurkishTokenizer obtains 0.7717 precision, 0.6348 recall, and 0.6966 boundary F1. It outperforms a neural Morpheus baseline using its official checkpoint (0.5443 F1), XLM-R (0.4281 F1), SentencePiece BPE-2000 (0.3607 F1), SentencePiece Unigram-2000 (0.3585 F1), BERTurk (0.2817 F1), mBERT (0.2801 F1), a character baseline (0.2882 F1), and a whole-word baseline (0.0000 F1). Paired bootstrap tests show large positive deltas over major subword baselines. We also report a 10,000-text exact round-trip test for the lossless API, a code-mixed foreign-root stress test, and probing results for root recovery and suffix-count prediction. The results show that lexicon-backed morphological segmentation is a strong and reproducible baseline for Turkish; the main remaining weakness is foreign-root code-mixing, where the current system reaches only 0.2597 F1 and should be improved in future releases.

Keywords: Turkish NLP; morphology-aware tokenization; language resources; human adjudication; benchmark; segmentation evaluation

1 Introduction

Tokenization is often treated as a preprocessing step, but for morphologically rich languages it can determine what information is visible to downstream models. Turkish is an agglutinative language in which stems combine with productive suffix chains. A tokenizer that splits words only according to frequency statistics may produce segments that are efficient for neural modeling but misaligned with linguistic boundaries. Conversely, a tokenizer that exposes roots and suffixes can support interpretability, probing, error analysis, and applications where morpheme-level information matters.

This paper describes NedoTurkishTokenizer and the accompanying Final1000 human-adjudicated benchmark. The system is not an external wrapper around another Turkish tokenizer. It is a self-contained tokenizer with a bundled Turkish lexicon, candidate-based segmentation, suffix heuristics, normalization, special-span handling, and a public token API. The contribution is deliberately scoped: this paper is about the tokenizer and its benchmark, not about a downstream language model.

The central evaluation question is simple: when a Turkish word has a human-adjudicated morpheme boundary set, how well does a tokenizer recover those boundaries? We evaluate

boundary precision, recall, and F1, exact boundary-set accuracy, over- and under-segmentation, label precision and recall for suffix labels, token-per-word rate, and stress behavior on foreign-root code-mixing. We compare against multilingual and Turkish subword baselines, character and whole-word baselines, and a neural Turkish morphology baseline.

The paper makes four contributions. First, it presents a practical morphology-aware Turkish tokenizer with a clean public token API and a separate lossless API for exact reconstruction. Second, it releases a human-adjudicated 1,000-item benchmark for Turkish morphological tokenization. Third, it provides a broad evaluation against neural, multilingual, BPE, Unigram, character, and whole-word baselines. Fourth, it reports ablations and stress tests that reveal both strengths and limitations, including a clear weakness on code-mixed foreign roots.

2 Related work

Turkish morphological analysis has a long history in rule-based and finite-state approaches. Mature tools such as Zemberek demonstrate the value of explicit Turkish morphology for analysis, normalization, and downstream NLP. The present work is aligned with that tradition, but it addresses a narrower and evaluation-oriented problem: tokenization boundaries and metadata for a public tokenizer API.

Subword tokenization methods such as byte-pair encoding and Unigram language-model tokenization are widely used in modern neural systems. These methods optimize compression or likelihood under training data distributions rather than linguistic boundary recovery. They are therefore appropriate baselines but not direct substitutes for a morphology-aware tokenizer when the target metric is human-adjudicated morpheme boundary agreement.

Transformer-era multilingual models such as mBERT and XLM-R use subword vocabularies learned across many languages. Turkish-specific models such as BERTurk improve coverage for Turkish text, yet their tokenization remains a subword engineering choice rather than an explicit morpheme-boundary objective. We therefore compare them as practical baselines but evaluate them under the same boundary-set metric.

Morpheus-style neural morphology models represent a different comparison point: rather than using frequency-based subword units, they aim to model morphological segmentation. For fairness, our Morpheus neural baseline was run with the official checkpoint and loaded cleanly, with zero missing and zero unexpected parameters.

3 System overview

NedoTurkishTokenizer exposes two conceptually separate interfaces. The normal `tokenize()` interface is the clean public token API. It emits tokens and metadata intended for downstream use. The `tokenize_lossless()` plus `detokenize()` interface is a separate reconstruction API. Lossless round-trip claims in this paper apply only to that lossless interface.

Each public token is represented as a dictionary containing at least a token surface, a token type, and a morphology position. Token types include roots, suffixes, foreign forms, punctuation, numbers, dates, units, URLs, mentions, hashtags, emoji, and related special spans. Suffix tokens may also carry canonical or label metadata used in label-level evaluation. The tokenizer handles normalization and special spans before candidate segmentation, then selects among candidate root-suffix analyses using lexicon and suffix heuristics.

The intended behavior is not to split every word aggressively. A good morphology-aware tokenizer should preserve known intact roots, split productive suffix chains when supported, avoid hallucinating suffix boundaries in unanalyzable forms, and expose special tokens such as URLs and numbers without corrupting their surface forms. This creates a precision-recall tradeoff: conservative splitting protects precision but can under-segment long or foreign-root forms, while aggressive splitting can improve recall but risks spurious boundaries.

4 Human-adjudicated benchmark

4.1 Gold set construction

The final benchmark contains 1,000 Turkish word items. It was built from two annotator streams plus adjudication. Items were accepted directly when annotators agreed on the boundary set; otherwise they were adjudicated. The final distribution is shown in Table 1.

Table 1: Final1000 gold-set composition.

Source	Count
Annotator-agreed	754
Previously adjudicated	50
Newly adjudicated	196
Total	1,000
Build issues	0

Each item contains the surface word, the gold segmentation, optional suffix labels, and an origin field. Build validation checks that the concatenated gold segments reconstruct the original surface up to case-insensitive comparison. The final build report contains zero bad rows.

4.2 Metrics

The primary metric is boundary F1. Given a gold segmentation and a predicted segmentation, each boundary is represented by its character offset inside the word. Precision is the fraction of predicted offsets that are gold offsets, recall is the fraction of gold offsets recovered, and F1 is their harmonic mean. This treats token surfaces consistently across methods, even when the predicted token strings differ.

We also report exact boundary-set accuracy, the percentage of items whose predicted boundary set exactly matches the gold set; over-segmentation rate, the percentage of items with at least one spurious boundary; under-segmentation rate, the percentage with at least one missing boundary; average tokens per word; and single-root rate on words whose gold analysis contains more than one segment. For suffix labels, we report label precision and label recall using canonical or suffix-label metadata when available.

5 Experimental setup

We evaluate NedoTurkishTokenizer against eight baseline families: whole-word, character, BERTurk WordPiece, XLM-R SentencePiece, mBERT WordPiece, SentencePiece BPE with 800, 2,000, and 8,000 vocabulary settings, SentencePiece Unigram with 800, 2,000, and 8,000 settings, and Morpheus neural segmentation. The main table reports the competitive or most relevant members for readability, while the full JSON result files retain all measured settings.

The Morpheus neural run used the official checkpoint and completed with `missing_keys=0` and `unexpected_keys=0`, which is important because a partially loaded checkpoint would make the comparison invalid. The run evaluated all 1,000 items and took 11.524 seconds in the recorded environment.

For statistical testing, we used paired bootstrap resampling over the 1,000 gold items. The reported confidence intervals are given as lower, median, and upper bootstrap estimates for the F1 delta. The recorded p-values are zero at the precision of the experiment output, so we report them as approximately zero rather than as a mathematically exact value.

6 Main results

Table 2 shows the main Final1000 boundary results. NedoTurkishTokenizer obtains 0.6966 F1, substantially above all measured baselines. The competitive external baseline is the Morpheus neural model at 0.5443 F1. Among general-purpose subword tokenizers, XLM-R is the competitive at 0.4281 F1. SentencePiece BPE and Unigram at 2,000 vocabulary size cluster around 0.36 F1. Character tokenization has perfect boundary recall but low precision, producing 0.2882 F1. Whole-word tokenization cannot predict internal morpheme boundaries and therefore has zero boundary F1.

Table 2: Boundary performance on the 1,000-item human-adjudicated benchmark.

Model	Precision	Recall	F1	Avg. tok/word
NedoTurkishTokenizer	0.7717	0.6348	0.6966	1.946
Morpheus neural	0.6023	0.4965	0.5443	1.948
XLM-R	0.4831	0.3843	0.4281	1.915
SentencePiece BPE-2000	0.2931	0.4687	0.3607	2.839
SentencePiece Unigram-2000	0.2970	0.4522	0.3585	2.751
Character	0.1684	1.0000	0.2882	7.831
BERTurk	0.4369	0.2078	0.2817	1.547
mBERT	0.2329	0.3513	0.2801	2.735
Whole-word	0.0000	0.0000	0.0000	1.000

Table 3 gives additional diagnostic metrics for NedoTurkishTokenizer. The exact boundary-set accuracy is 64.70%. The tokenizer over-segments 16.70% of cases and under-segments 30.90% of cases, confirming that remaining errors are more often missed boundaries than spurious boundaries. Label precision is 0.5318 and label recall is 0.4129, so suffix labeling remains weaker than boundary detection.

Table 3: NedoTurkishTokenizer diagnostic metrics on Final1000.

Metric	Value
Boundary precision	0.7717
Boundary recall	0.6348
Boundary F1	0.6966
Exact boundary-set accuracy	64.70%
Average tokens per word	1.946
Single-root rate	15.20%
Over-segmentation rate	16.70%
Under-segmentation rate	30.90%
Label precision	0.5318
Label recall	0.4129

7 Statistical significance

Table 4 reports paired bootstrap comparisons. The deltas are large and consistently positive. NedoTurkishTokenizer exceeds XLM-R by 0.2685 F1, BERTurk by 0.4149 F1, mBERT by 0.4165 F1, and SentencePiece Unigram-2000 by 0.3381 F1. The confidence intervals do not cross zero.

Table 4: Paired bootstrap F1 deltas on Final1000. Confidence intervals are lower, median, upper.

Comparison	Delta	CI	p
Nedo vs. XLM-R	+0.2685	[0.2329, 0.2693, 0.3035]	approx. 0
Nedo vs. BERTurk	+0.4149	[0.3758, 0.4154, 0.4559]	approx. 0
Nedo vs. mBERT	+0.4165	[0.3824, 0.4153, 0.4465]	approx. 0
Nedo vs. SP-Unigram-2000	+0.3381	[0.3030, 0.3379, 0.3742]	approx. 0

8 Ablations

Table 5 reports human-gold ablations. Removing the domain vocabulary does not change the aggregate result on this 1,000-item set, suggesting that the present benchmark is not strongly domain-vocabulary-driven. Removing the lexicon collapses F1 to 0.0965, showing that lexicon support is central to the system. Reducing segmentation depth to 1 also hurts substantially, while depth 3 slightly improves the aggregate F1 to 0.7009 in this evaluation. Because the difference between full and depth 3 is small, this should be treated as an engineering signal for further tuning rather than as a new final claim.

Table 5: Human-gold ablation on the 1,000-item benchmark.

Condition	Precision	Recall	F1	Avg. tok/word	Single
Full	0.7717	0.6348	0.6966	1.946	15.20%
No domain vocabulary	0.7717	0.6348	0.6966	1.946	15.20%
No lexicon	0.8082	0.0513	0.0965	1.073	65.70%
Depth = 3	0.7824	0.6348	0.7009	1.933	15.20%
Depth = 1	0.8765	0.3270	0.4763	1.429	27.00%

9 Round-trip reconstruction

The tokenizer provides a separate lossless interface for exact reconstruction. Using `tokenize_lossless()` followed by `detokenize()`, the system reconstructs 10,000 out of 10,000 sampled texts exactly. The run used 500 characters per text and completed with zero mismatches. This claim is intentionally limited to the lossless interface. It should not be read as a claim that the normal `tokenize()` API preserves every original whitespace or raw surface detail; the normal API is the clean public tokenization interface.

Table 6: Lossless API round-trip test.

Metric	Value
Texts	10,000
Exact reconstructions	10,000
Mismatches	0
Characters per text	500
Runtime	161.809 seconds

10 Morphological probing

We also evaluate whether predicted tokenization exposes useful morphological structure beyond boundary F1. Table 7 reports root recovery, suffix-count accuracy, suffix-count mean absolute error, and exact boundary accuracy. The overall root recovery accuracy is 0.7120, suffix-count accuracy is 0.7360, and boundary-exact accuracy is 0.6470. Performance is stronger on short words than on long words. For words shorter than 10 characters, root accuracy is 0.7401 and boundary-exact accuracy is 0.6953. For words of length 10 or more, root accuracy drops to 0.6240 and boundary-exact accuracy to 0.4959.

Table 7: Morphological probing on Final1000.

Metric	Value
Root recovery accuracy	0.7120
Suffix-count accuracy	0.7360
Suffix-count MAE	0.3620
Boundary-exact accuracy	0.6470
Short words root accuracy	0.7401
Short words boundary-exact accuracy	0.6953
Long words root accuracy	0.6240
Long words boundary-exact accuracy	0.4959

11 Code-mixed foreign-root stress test

The clearest weakness is foreign-root code-mixing. On 240 constructed code-mixed and foreign-root cases, the tokenizer reaches 0.5882 precision, 0.1667 recall, and 0.2597 F1, with an 81.67% single-root rate. Typical failures include treating forms such as *meetingler*, *pluginde*, and *driver-den* as unsplit foreign-root tokens rather than foreign root plus Turkish suffix. This is a limitation, not a minor noise source: the gap between 0.6966 Final1000 F1 and 0.2597 code-mix F1 indicates that the current suffix-splitting strategy is too conservative for foreign roots.

Future versions should add a dedicated foreign-root suffix analyzer, improve vowel-harmony-aware suffix matching after non-Turkish roots, and evaluate code-mixed corpora rather than only constructed stress items. Until this is fixed, the tokenizer should be described as strong on standard Turkish morphological tokenization but weak on foreign-root code-mixed segmentation.

Table 8: Code-mixed / foreign-root stress test.

Metric	Value
Cases	240
Precision	0.5882
Recall	0.1667
F1	0.2597
Single-root rate	81.67%

12 Error analysis

The aggregate diagnostics show that under-segmentation is the dominant error mode. This is expected for a lexicon-backed system that avoids splitting uncertain forms. The upside is

relatively high precision; the downside is missed suffix boundaries, especially in long forms and foreign-root forms. Label-level scores are lower than boundary scores, indicating that after a boundary is found, canonical suffix labeling remains an additional source of error.

The long-word probe provides a second view of the same issue. Long Turkish words are more likely to contain multiple suffixes, derivational chains, proper-name effects, or surface alternations. Exact recovery therefore requires not just detecting a suffix but selecting the correct chain depth and preserving the correct root. The depth ablation suggests that allowing more than one suffix is necessary, but the precise search depth and candidate ranking still need tuning.

The code-mixed stress test reveals a separate source of error. Foreign roots are often preserved as unanalyzed units, which is reasonable for unknown words in general but harmful when Turkish suffixes are productively attached to English or other foreign stems. This issue should be highlighted in the limitations and should motivate a separate release milestone.

13 Availability and reproducibility

The final evaluation artifacts are stored under the TRUBA project directory:

`/arf/scratch/egitimg16u1/nedoturkishtokenizer/reports/q1_worldclass_final/`

The core files are `full_gold_1000.jsonl`, `final1000_nedo_metrics.json`, `final1000_external_baselines.j` `morpheus_neural_1000.json`, `human_gold_ablation_1000.json`, `paired_bootstrap_1000.json`, `roundtrip_10k.json`, `codemix_foreign_240.json`, `morphological_probe_1000.json`, and `full_gold_build_`

For publication, the tokenizer source code should be released in a public GitHub repository, packaged for `pip`, and mirrored on Hugging Face if appropriate. The Final1000 benchmark should be released as a dataset with a persistent DOI, for example through Zenodo. The paper should include stable release identifiers rather than only TRUBA paths.

14 Limitations

This work has several limitations. First, the benchmark contains 1,000 items. It is adjudicated and useful for controlled evaluation, but it is still modest in size. Second, the benchmark is word-level rather than sentence-level, so it evaluates segmentation boundaries and labels rather than downstream sentence understanding. Third, the code-mixed foreign-root stress test is poor, with only 0.2597 F1; this limitation must be visible in the abstract, results, and error analysis. Fourth, the lossless reconstruction claim applies only to `tokenize_lossless()` plus `detokenize()`, not to the normal clean token API. Fifth, label-level evaluation is weaker than boundary evaluation and needs additional annotation consistency checks.

15 Extended benchmark protocol

Final1000 is intentionally evaluated as a word-level boundary benchmark. This keeps the target precise: a system receives a surface word and is scored according to whether it recovers the adjudicated internal boundary offsets. The benchmark does not require a complete sentence-level morphological parse, contextual disambiguation, or semantic interpretation. This design makes it useful for tokenizer evaluation because tokenizers are commonly evaluated before downstream context is available.

The gold set contains three origin groups. Annotator-agreed items are those where the two annotation streams agreed on the boundary set. Previously adjudicated items come from an earlier adjudication round. Newly adjudicated items are the missing disagreement cases resolved in the final build. The final build contains zero surface-mismatch or unresolved rows. This is

important because even a small number of malformed rows can distort boundary precision and recall.

The annotation representation is deliberately simple: segments are stored as ordered strings whose concatenation reconstructs the surface word under the benchmark comparison rule. Optional labels describe suffix identity when available. Boundary evaluation and label evaluation are separated so that systems can be compared even when their label inventories differ.

16 Extended reproducibility package

The release staging package is organized into frozen results, dataset files, package checks, release cards, validation artifacts, scripts, and paper files. The frozen results directory contains the JSON files used for the claims in this manuscript together with SHA256 checksums. The dataset directory contains a JSONL release file, a CSV mirror, a HuggingFace-style dataset card, and Zenodo metadata. The package directory contains an import and sample-tokenization sanity check. The validation directory contains error examples, long-word analysis, label-error summaries, segmentation error inventory, and foreign-root stress summaries.

Public release still requires external account actions. The GitHub repository, pip package, HuggingFace dataset, Zenodo DOI, and arXiv preprint URL must be created outside TRUBA and inserted into the manuscript before final submission. These identifiers are not fabricated in this draft.

17 Extended error analysis

The aggregate metrics show that under-segmentation is the main boundary error. This is consistent with a conservative lexicon-backed tokenizer: it avoids many false splits, but misses boundaries when a root is unknown, when a suffix chain is unusually deep, when surface alternation is difficult, or when the word is a foreign-root code-mixed form.

Table 9 lists representative exact matches, while Table 10 lists representative boundary errors.

Table 9: Representative exact boundary matches from the validation artifact.

Word	Gold	Prediction	Mode
öğretmenlerimizin	öğretmen+ler+imiz+in	öğretmen+ler+imiz+in	exact
öğrencilerimizin	öğrenci+ler+imiz+in	öğrenci+ler+imiz+in	exact
görüşmelerimizi	görüşme+ler+imiz+i	görüşme+ler+imiz+i	exact
kademelerimizde	kademe+ler+imiz+de	kademe+ler+imiz+de	exact
Üreticilerimize	Üretici+ler+imiz+e	Üretici+ler+imiz+e	exact
kurulabilirler	kurul+abil+ir+ler	kurul+abil+ir+ler	exact
buluşmalarını	buluşma+lar+ı+nı	buluşma+lar+ı+nı	exact
derslerimizin	ders+ler+imiz+in	ders+ler+imiz+in	exact
programlarına	program+lar+ın+a	program+lar+ın+a	exact
yaklaşmasının	yaklaş+ma+sı+nın	yaklaş+ma+sı+nın	exact

18 Origin-wise and length-wise diagnostics

Adjudicated items are expected to be more difficult than annotator-agreed items, because they come from disagreement cases. Table 11 summarizes performance by origin group. Length is another major difficulty factor. In the generated validation analysis, words shorter than 10 characters and words of length 10 or greater show a clear gap.

Table 10: Representative boundary errors from the validation artifact.

Word	Gold	Prediction	Mode
yakalayacağını	yakala+yacağ+ı+nı	yakalay+a+ca+ğı+nı	over+under
teşkilatında	teşkilat+ı+nda	teşkil+a+tın+da	over+under
kullanılan	kullan+ıl+an	kul+la+nı+lan	over+under
şirketinin	şirket+i+nin	şirk+e+tin+in	over+under
gençleştirmesinde	genç+leş+tir+me+si+nde	gençleştir+me+sin+de	over+under
GERÇEKLEŞTİRDİK	GERÇEK+LEŞ+TİR+Dİ+K	GERÇEKLEŞTİRDİK	under
belirtilerinin	belirti+ler+i+nin	belirtil+e+ri+nin	over+under
ilgilenmeyince	ilgilen+me+yince	ilgilenmey+in+ce	over+under
katılamazlar	katıl+a+ma+z+lar	katılamazlar	under
kıyılarında	kıyı+lar+ı+nda	kıyıl+ar+ın+da	over+under
mağduriyeti	mağduriyet+i	mağdur+i+ye+ti	over+under
unutmayınız	unut+ma+yınız	unutmay+ın+ız	over+under
yatırdığını	yatır+dığ+ı+nı	yatırd+ı+ğı+nı	over+under
ödemesine	öde+me+si+ne	ödem+e+sin+e	over+under
Kariyeri	Kariyer+i	Kar+i+ye+ri	over+under

Table 11: Boundary performance by gold-set origin.

Origin	Cases	F1	Exact	Under
Annotator-agreed	754	0.8224	80.50%	15.65%
Previously adjudicated	50	0.4412	18.00%	76.00%
Newly adjudicated	196	0.4806	15.82%	78.06%

19 Extended label diagnostics

Boundary detection and suffix-label recovery are related but not identical. A system can place a correct boundary while still assigning an incomplete or different canonical suffix label. Table 13 lists the largest label-level error entries from the generated validation artifact. These entries should guide future canonical-label cleanup and annotation consistency work.

20 Extended code-mix discussion

The code-mixed foreign-root stress test is intentionally separated from Final1000. It asks whether foreign roots with Turkish suffixes are segmented into a foreign root plus productive Turkish suffixes. The current tokenizer frequently keeps such forms as single tokens. This explains the high single-root rate and the low recall. The future fix should be a dedicated foreign-root suffix analyzer, not a general increase in aggressive splitting. A general increase in aggressive splitting could improve recall but damage precision on ordinary unknown words.

21 Submission-readiness notes

The manuscript is currently a locally compiled LRE-preparation draft. It is not yet a final submission package because public identifiers are still missing. Before submission, the authors should replace local TRUBA paths with public repository, package, dataset, DOI, and preprint URLs; transfer the manuscript to the official Springer/LRE template if required; complete declarations; and verify that the dataset license is compatible with public release.

Table 12: Length-wise diagnostic summary generated from Final1000.

Subset	Cases	F1	Exact	Over	Under
Short < 10	758	0.6882	69.53%	12.93%	26.91%
Long ≥ 10	242	0.7072	49.59%	28.51%	43.39%

Table 13: Largest suffix-label error entries.

Label	Hit	FP	FN	Precision	Recall
ACC	17	167	2	0.0924	0.8947
P3SG	0	0	112	0.0000	0.0000
GEN	3	105	0	0.0278	1.0000
PL	64	61	6	0.5120	0.9143
P3	0	51	0	0.0000	0.0000
NEG	3	31	6	0.0882	0.3333
DAT	58	28	5	0.6744	0.9206
PTCP	0	0	33	0.0000	0.0000
VN	0	0	27	0.0000	0.0000
PASS/REFL	0	0	23	0.0000	0.0000
LOC	50	10	12	0.8333	0.8065
ADJ	0	0	21	0.0000	0.0000
CV	0	0	19	0.0000	0.0000
P3SG/ACC	0	0	17	0.0000	0.0000
PAST	15	12	5	0.5556	0.7500
COP	0	0	16	0.0000	0.0000
2SG	0	14	0	0.0000	0.0000
NMLZ	0	0	14	0.0000	0.0000

22 Conclusion

NedoTurkishTokenizer provides a strong morphology-aware tokenizer and an accompanying human-adjudicated benchmark for Turkish tokenization. On Final1000 it obtains 0.6966 boundary F1, outperforming neural, multilingual, BPE, Unigram, character, and whole-word baselines under the same boundary metric. The system also passes a 10,000-text exact round-trip test through its separate lossless API. The evidence supports submission as a language-resource and tokenizer-evaluation paper, especially for a venue centered on language resources and evaluation. The most important remaining work before public release is not to hide the weak code-mixed result but to document it clearly, release the dataset and code with persistent identifiers, and plan a follow-up improvement for foreign-root suffix splitting.

Declarations

Manuscript note. This TRUBA manuscript variant includes author information and local reproducibility paths.

Funding. No external funding statement has been provided in the TRUBA artifacts. This must be completed before journal submission if applicable.

Competing interests. No competing-interest statement has been provided in the TRUBA artifacts. This must be completed before journal submission.

Data availability. The Final1000 benchmark is currently available as a TRUBA artifact. For submission, it should be released publicly with a persistent DOI and referenced here.

Code availability. The tokenizer source is present in the TRUBA project directory. For submission, a public repository URL and package version should be inserted here.

Ethics statement. The benchmark consists of Turkish word-level segmentation annotations. If any source words were drawn from private or restricted data, the source and permission status must be documented before submission.

Lossless roundtrip validation. A separate lossless roundtrip check was run on 10,000 cases (10000 cases). The check produced 0 mismatches. This result applies to `tokenize_lossless()` followed by `detokenize()`, and should not be interpreted as a guarantee for every normal `tokenize()` call.

Limitations

The reported results are limited to the evaluation sets described in this paper. The main Final1000 set measures morphology-aware tokenization on curated Turkish examples. Performance is weaker on code-mixed foreign-root forms with Turkish suffixes, and this portion of the benchmark should be expanded in future work. The lossless roundtrip result applies to the explicit lossless API rather than to every normal tokenization call.

Data and Code Availability

The source code of NedoTurkishTokenizer is available at: `https://github.com/NMSOfficial/NedoTurkishTokenizer`.

The Final1000 evaluation dataset is available on Hugging Face: `https://huggingface.co/datasets/Ethosoft/NedoTurkishTokenizer-Final1000`.

The archived release is available through Zenodo: `https://doi.org/10.5281/zenodo.21274980`.

References

[1] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*.

[2] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzman, F., Grave, E., Ott, M., Zettlemoyer, L., and Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. In *Proceedings of ACL*.

[3] Kudo, T. and Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of EMNLP: System Demonstrations*.

[4] Sennrich, R., Haddow, B., and Birch, A. (2016). Neural machine translation of rare words with subword units. In *Proceedings of ACL*.

[5] Akin, A. A. and Akin, M. D. (2007). Zemberek, an open source NLP framework for Turkic languages. *Structure*, 10, 1-5.

[6] Coltekin, C., Dogruoz, A. S., and Cetinoglu, O. (2022). Resources for Turkish natural language processing: A critical survey. *Language Resources and Evaluation*.